

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении лабораторной работы №4
По дисциплине: «Программирование микроконтроллеров»
На тему: «Таймер и прерывания»

Выполнил студент
группы 5130201/40002

_____ Семенов И. А.

Проверила
к.т.н. доцент

_____ Верובה Н. М.

«_____» _____ 2026г.

Содержание

1	Цель работы	3
2	Постановка задачи	3
3	Выполнение работы	3
3.1	Функция задержки	3
3.2	Инициализация GPIO	3
3.3	Настройка таймера TIM2	4
3.4	Обработчик прерывания таймера	4
3.5	Настройка внешнего прерывания EXTI0 (WAKEUP)	5
3.6	Обработчик EXTI0	5
4	Работа с осциллографом	6
5	Вывод	10
	Приложение. Полный текст программы	11

1 Цель работы

Ознакомиться с основными методами формирования заданных интервалов времени и перевода микроконтроллера в режим пониженного энергопотребления. Закрепить навыки работы с осциллографом и оценочной платой MCBSTM32F200 в качестве измерительного генератора.

2 Постановка задачи

Разработать программу для микроконтроллера STM32F200, которая при помощи таймера формирует периодическое попеременное включение и выключение светодиодов PG6 и PG7 с заданными временными характеристиками. При нажатии на кнопку WAKEUP программа должна переводить микроконтроллер в спящий режим, а при ее отпуске — пробуждать микроконтроллер.

3 Выполнение работы

В среде Keil μ Vision5 был создан проект, в котором реализовано управление светодиодами с использованием таймера и обработка внешнего прерывания. Ниже приведены ключевые фрагменты программы и пояснения.

3.1 Функция задержки

Для создания программной задержки используется простая пустая петля. Переменная цикла объявлена с квалификатором `volatile`, чтобы компилятор не оптимизировал цикл.

Листинг 1: Delay function (milliseconds)

```
1 void delay_ms(uint32_t milliseconds) {  
2     volatile uint32_t i;  
3     for (i = 0; i < (milliseconds * 4200U); i++) { __NOP(); }  
4 }
```

3.2 Инициализация GPIO

Включается тактирование портов GPIOA и GPIOG, настраиваются выходы PG6 и PG7 как выходы, а PA0 как вход для кнопки WAKEUP.

Листинг 2: GPIO initialization

```
1 RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN | RCC_AHB1ENR_GPIOGEN;
2
3 /* PG6, PG7 */
4 GPIOG->MODER &= ~((3U << (6*2)) | (3U << (7*2)));
5 GPIOG->MODER |= (1U << (6*2)) | (1U << (7*2));
6
7 /* PA0 (WAKEUP) */
8 GPIOA->MODER &= ~(3U << (0*2));
```

3.3 Настройка таймера TIM2

Таймер TIM2 настраивается с предделителем и автоперезагрузкой. После настройки включается прерывание по обновлению, задается приоритет обработчика и разрешается работа таймера.

Листинг 3: TIM2 setup

```
1 RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
2
3 TIM2->PSC = 119990U;
4 TIM2->ARR = 249U;
5
6 TIM2->CR1 |= TIM_CR1_ARPE | TIM_CR1_URS;
7 TIM2->DIER |= TIM_DIER_UIE;
8 TIM2->CR1 |= TIM_CR1_CEN;
9
10 NVIC_SetPriority(TIM2_IRQn, 0);
11 NVIC_EnableIRQ(TIM2_IRQn);
```

3.4 Обработчик прерывания таймера

При каждом переполнении таймера выполняется переключение светодиодов PG6 и PG7. После обработки события сбрасывается флаг прерывания таймера.

Листинг 4: TIM2 IRQ Handler

```
1 void TIM2_IRQHandler(void) {
2     if (TIM2->SR & TIM_SR_UIF) {
3         GPIOG->ODR ^= (1U << 6) | (1U << 7);
4         TIM2->SR &= ~TIM_SR_UIF;
5     }
6 }
```

3.5 Настройка внешнего прерывания EXTI0 (WAKEUP)

Включается тактирование SYSCFG, линия EXTI0 привязывается к PA0, разрешается прерывание по обоим фронтам и настраивается NVIC.

Листинг 5: EXTI0 setup

```
1 RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
2
3 SYSCFG->EXTICR[0] &= ~SYSCFG_EXTICR1_EXTI0;
4 SYSCFG->EXTICR[0] |= SYSCFG_EXTICR1_EXTI0_PA;
5
6 EXTI->IMR |= EXTI_IMR_MR0;
7 EXTI->RTSR |= EXTI_RTSR_TR0;
8 EXTI->FTSR |= EXTI_FTSR_TR0;
9
10 NVIC_SetPriority(EXTI0_IRQn, 0);
11 NVIC_EnableIRQ(EXTI0_IRQn);
```

3.6 Обработчик EXTI0

Обработчик реализует подавление дребезга, переключение режимов сна (SLEEPDEEP, SLEEPONEXIT) и изменение типа срабатывания EXTI для корректного входа в сон и пробуждения.

Листинг 6: EXTI0 IRQ Handler

```
1 void EXTI0_IRQHandler(void) {
2     if (EXTI->PR & EXTI_PR_PRO) {
3         delay_ms(100); /* debounce */
4
5         if (!sleep_mode) {
6             /* Enter deep sleep on exit from IRQ */
7             SCB->SCR |= SCB_SCR_SLEEPDEEP_Msk;
8             SCB->SCR |= SCB_SCR_SLEEPONEXIT_Msk;
9
10            /* Switch EXTI to falling edge for wake-up */
11            EXTI->RTSR &= ~EXTI_RTSR_TR0;
12            EXTI->FTSR |= EXTI_FTSR_TR0;
13            sleep_mode = true;
14        } else {
15            /* Exit sleep mode */
16            SCB->SCR &= ~SCB_SCR_SLEEPONEXIT_Msk;
17            SCB->SCR &= ~SCB_SCR_SLEEPDEEP_Msk;
18
19            /* Restore EXTI to trigger on rising edge */
20            EXTI->RTSR |= EXTI_RTSR_TR0;
21            EXTI->FTSR &= ~EXTI_FTSR_TR0;
22            sleep_mode = false;
23        }
24
25        /* Clear EXTI pending flag */
26        EXTI->PR = EXTI_PR_PRO;
27    }
28 }
```

4 Работа с осциллографом

При исследовании сигнала на выходе микроконтроллера был получен прямоугольный сигнал. По экрану осциллографа период сигнала составил примерно $T = 1.9$ с, что соответствует частоте около $f = 0.53$ Гц. По встроенным измерениям осциллографа были определены времена нарастания и спада сигнала, после чего фронт и спад были рассмотрены на растянутой временной развертке.

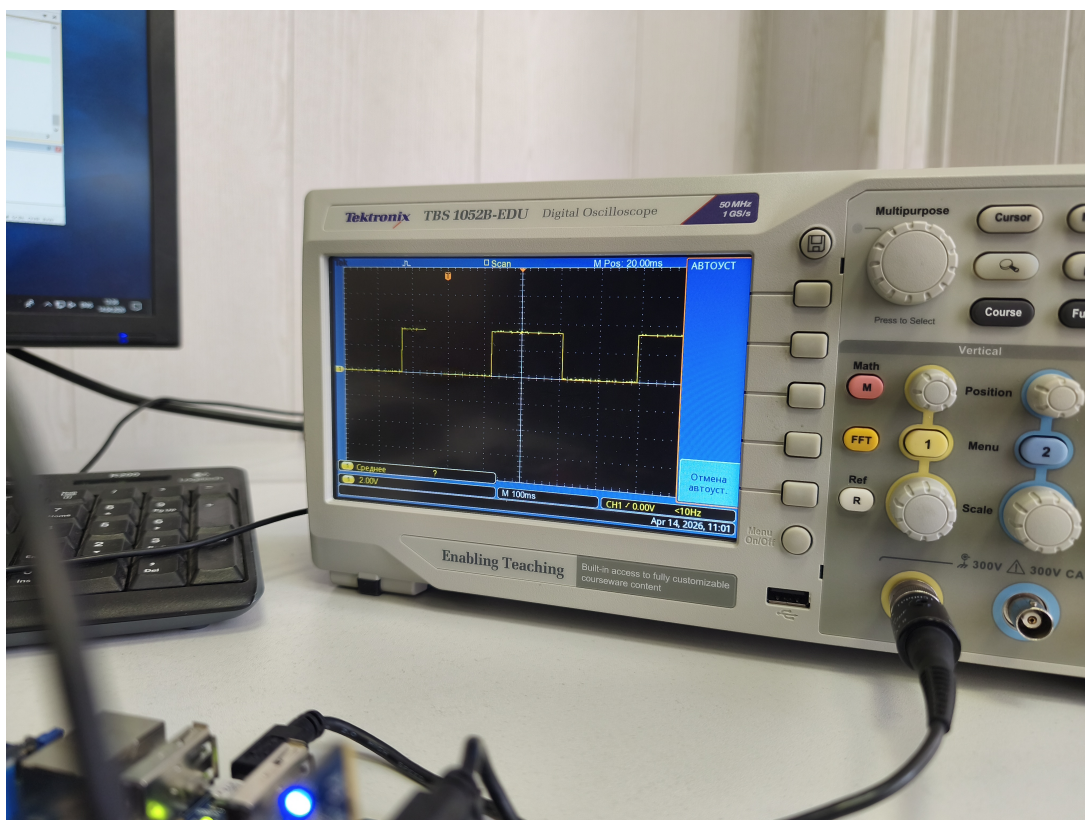


Рис. 1: Осциллограмма прямоугольного сигнала на выходе микроконтроллера.

На фотографии 2 показан результат автоматического измерения времени нарастания. Измеренное время нарастания составило $t_r = 10.16$ нс, размах сигнала составил примерно $V_{pp} = 3.64$ В.

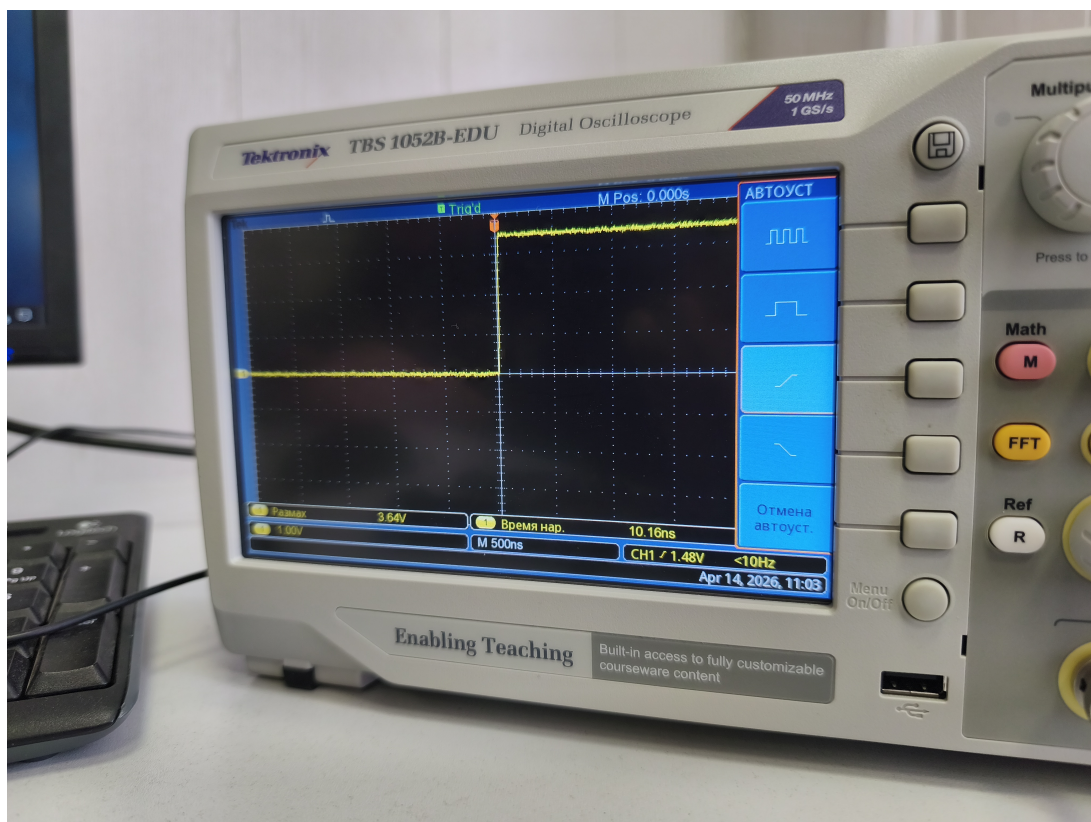


Рис. 2: Явное измерение времени нарастания сигнала.

На фотографии 3 показан результат автоматического измерения времени спада. Измеренное время спада равно $t_f = 10.11$ нс, размах сигнала составил примерно $V_{pp} = 3.60$ В.

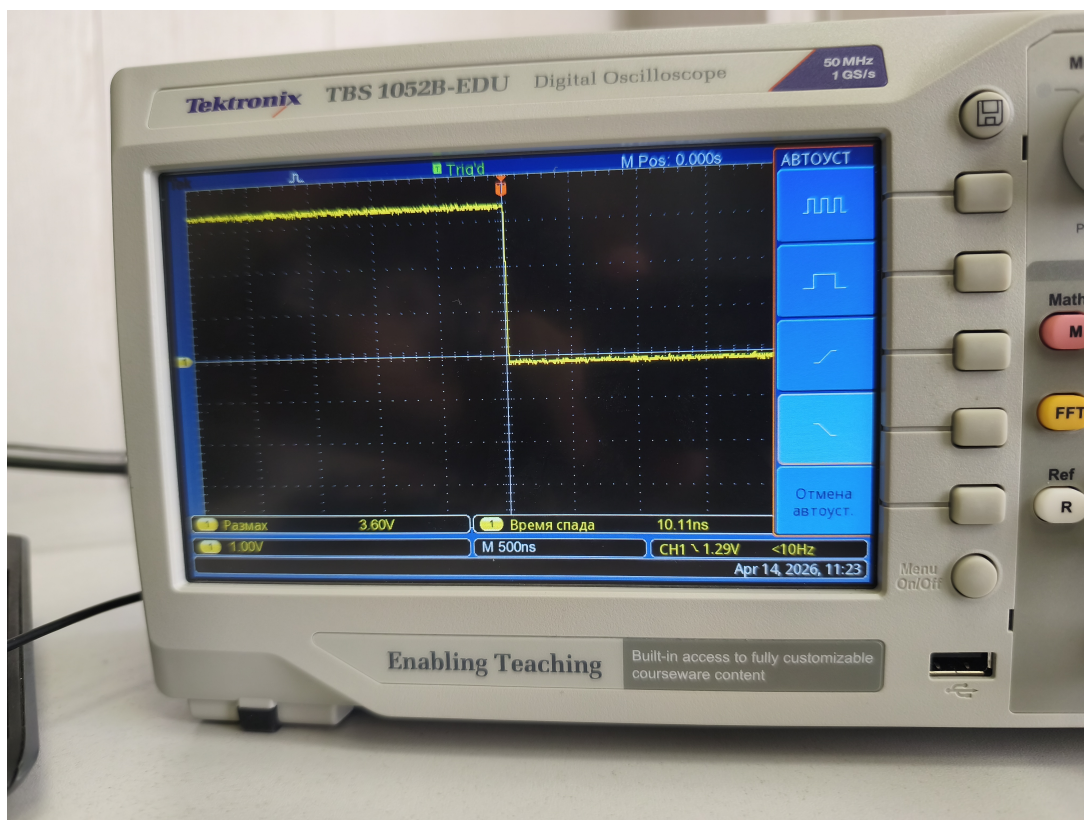


Рис. 3: Явное измерение времени спада сигнала.

После явного измерения был растянут фронт сигнала для визуальной проверки формы переходного процесса.



Рис. 4: Растянутое изображение фронта сигнала.

Также был растянут спад сигнала. На экране осциллографа дополнительно отображалось значение $t_f = 10.10$ нс.

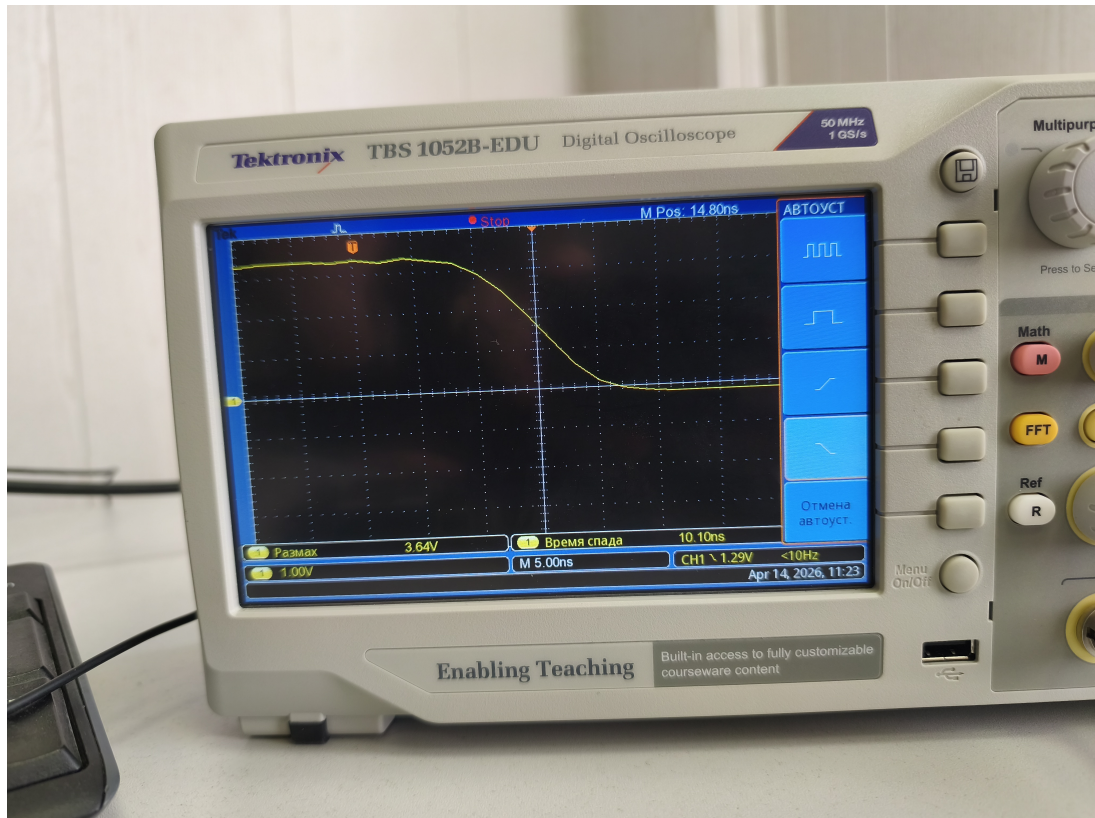


Рис. 5: Растянутое изображение спада сигнала.

5 Вывод

В ходе выполнения лабораторной работы была разработана программа для микроконтроллера STM32F200, реализующая формирование временных интервалов при помощи таймера TIM2 и обработку внешнего прерывания от кнопки WAKEUP. Были изучены настройка таймера, обработка его прерывания, работа с EXTI и перевод микроконтроллера в спящий режим. С помощью осциллографа были измерены параметры выходного сигнала: период около 1.9 с, частота около 0.53 Гц, размах около 3.64 В, время нарастания 10.16 нс и время спада 10.11 нс.

Приложение. Полный текст программы

```
1 #include "stm32f2xx.h"
2 #include <stdbool.h>
3
4 static volatile bool sleep_mode = false;
5
6 void delay_ms(uint32_t milliseconds)
7 {
8     volatile uint32_t i;
9     for (i = 0; i < (milliseconds * 4200U); i++) {
10         __NOP();
11     }
12 }
13
14 void TIM2_IRQHandler(void)
15 {
16     if (TIM2->SR & TIM_SR_UIF) {
17         GPIOG->ODR ^= (1U << 6) | (1U << 7);
18         TIM2->SR &= ~TIM_SR_UIF;
19     }
20 }
21
22 void EXTI0_IRQHandler(void)
23 {
24     if (EXTI->PR & EXTI_PR_PRO) {
25         delay_ms(100);
26
27         if (!sleep_mode) {
28             SCB->SCR |= SCB_SCR_SLEEPDEEP_Msk;
29             SCB->SCR |= SCB_SCR_SLEEPONEXIT_Msk;
30             EXTI->RTSR &= ~EXTI_RTSR_TR0;
31             EXTI->FTSR |= EXTI_FTSR_TR0;
32             sleep_mode = true;
33         } else {
34             SCB->SCR &= ~SCB_SCR_SLEEPONEXIT_Msk;
35             SCB->SCR &= ~SCB_SCR_SLEEPDEEP_Msk;
36             EXTI->RTSR |= EXTI_RTSR_TR0;
37             EXTI->FTSR &= ~EXTI_FTSR_TR0;
38             sleep_mode = false;
39         }
40
41         EXTI->PR = EXTI_PR_PRO;
42     }
43 }
44
45 int main(void)
46 {
47     RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN | RCC_AHB1ENR_GPIOGEN;
48     RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
49     RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
50 }
```

```

51     GPIOG->MODER &= ~((3U << (6 * 2)) | (3U << (7 * 2)));
52     GPIOG->MODER |= ((1U << (6 * 2)) | (1U << (7 * 2)));
53     GPIOA->MODER &= ~(3U << (0 * 2));
54
55     TIM2->PSC = 119990U;
56     TIM2->ARR = 249U;
57     TIM2->CR1 |= TIM_CR1_ARPE | TIM_CR1_URS;
58     TIM2->DIER |= TIM_DIER_UIE;
59     TIM2->CR1 |= TIM_CR1_CEN;
60
61     SYSCFG->EXTICR[0] &= ~SYSCFG_EXTICR1_EXTIO;
62     SYSCFG->EXTICR[0] |= SYSCFG_EXTICR1_EXTIO_PA;
63     EXTI->IMR |= EXTI_IMR_MR0;
64     EXTI->RTSR |= EXTI_RTSR_TR0;
65     EXTI->FTSR |= EXTI_FTSR_TR0;
66
67     NVIC_SetPriority(TIM2_IRQn, 0);
68     NVIC_EnableIRQ(TIM2_IRQn);
69     NVIC_SetPriority(EXTIO_IRQn, 0);
70     NVIC_EnableIRQ(EXTIO_IRQn);
71
72     while (1) {
73         __WFI();
74     }
75 }

```